Fitting occupancy models

Analysing Ecological Data with Detection Error

Péter Sólymos

2026-04-21

Table of contents

Preamble 1

Simulating occupancy data 2

Simulating multiple visits 4

Fitting occupancy models 7

Maximum likelihood estimator 9

Field data analysis 12

Preamble

```
suppressPackageStartupMessages({
    library(unmarked)
    library(detect)
    library(dplyr)
    library(ggplot2)
    theme_set(theme_light())
})
set.seed(1234)
```

Simulating occupancy data

Let's see how we can use the Bernoulli distribution and the `rbinom` function to simulate occupancy data.

Let us sample $n = 50$ locations and observe the occupancy status of Ovenbirds at each of the locations:

- The true occupancy status at the ith location is W_i.
- The probability of occupancy is $P(W_i = 1) = \phi$.

The true occupancy probability ϕ is denoted with `phi.true`:

```
n <- 50
phi.true <- 0.4

W <- rbinom(n = n, size = 1, prob = phi.true)

table(W = W)
```

```
W
 0  1
31 19
```

There are many reasons why we might not observe the true occupancy status W. When the data that we collect (Y) is not the true occupancy status W, we say that there is detection error. It can manifest the following way:

We can capture detection error also as a probability p which is the probability of detecting the species when present ($W_i = 1$).

- If the species is absent ($W_i = 0$), we will always observe 0, thus $Y_i = 0$: $P(Y_i = 0|W_i = 0) = 1$.
- If $W_i = 1$, we might observe 0 (missed all the birds) or 1:
  - species detected when present, $P(Y_i = 1|W_i = 1) = p$
  - species not detected when present, $P(Y_i = 0|W_i = 1) = 1 - p$

Let's simulate what we observe, the `Y` vector. We use the `rbinom()` function with `n = n` as before, but the size argument is not a fixed 1, but the true status `W`. When `W` is 0, the function will always return 0, when `W` is 1, the function will reyrun 1 or 0 according the the detection probability p, denoted as `p.true`:

```r
p.true <- 0.6
Y <- rbinom(n = n, size = W, prob = p.true)

table(W = W, Y = Y)
```

```
   Y
W    0  1
  0 31  0
  1  5 14
```

The cross-tabulation show how many times we observed 1's vs 0's when the true status was 1.

Using the observations as if those would represent the true status is called the *naive* approach. The naive approach assume that there is no observation error:

```r
mean(Y)
```

```
[1] 0.28
```

But we see that the mean of `Y` is quite far from `phi.true`.

Fitting a logistic regression (aka Binomial GLM) to the data yields the same result:

```r
m1 <- glm(Y ~ 1, family = binomial)
coef(m1)
```

```
(Intercept)
 -0.9444616
```

```r
plogis(coef(m1))
```

```
(Intercept)
       0.28
```

The coefficient for the intercept is the maximum likelihood estimate on the logit scale. To transform that to the $0-1$ probability scale, we can use the `plogis()` function that implements the inverse logistic transformation.

The unobserved `W` variable is often called a *latent* variable. It is latent in the sense that it cannot be directly observed. We need to find ways to:

- use survey design to minimize the detection error, or
- use statistical models to correct for detection error, or
- the combination of the 2.

Simulating multiple visits

A feature of multiple-visit methods is that we visit the same site T times. Each visit ($t = 1, \dots, T$) gives us a different observation, Y_{it}.

Key assumptions are the following:

- the visits are independent of each other
- the true status W stays the same over the visits ($W_i = W_{it}$)

We can generalize the simulation code to any number of `T` visits as:

```
T <- 5
Y <- matrix(NA, n, T)
for (t in 1:T) {
    Y[, t] <- rbinom(n = n, size = W, prob = p.true)
}
```

A more concise way of writing the above code without a loop is:

```
Y <- replicate(T, rbinom(n = n, size = W, prob = p.true))
```

If we inspect any row (site) from the `Y` matrix, we see a series of 1's and 0's for sites where the species is present. This is called the *detection history*.

```
data.frame(
    W = head(W[W > 0]),
    Visit = head(Y[W > 0, ])
)
```

```
  W Visit.1 Visit.2 Visit.3 Visit.4 Visit.5
1 1       1       1       1       1       0
2 1       1       0       1       1       0
3 1       0       1       1       1       1
4 1       1       0       0       1       0
5 1       0       1       1       0       1
6 1       1       1       0       0       1
```

The detection history is all 0's for sites where the species is absent:

```
data.frame(
    W = head(W[W == 0]),
    Visit = head(Y[W == 0, ])
)
```

```
  W Visit.1 Visit.2 Visit.3 Visit.4 Visit.5
1 0       0       0       0       0       0
2 0       0       0       0       0       0
3 0       0       0       0       0       0
4 0       0       0       0       0       0
5 0       0       0       0       0       0
6 0       0       0       0       0       0
```

We can compare the maximum of the observed values over the visits at each site:

```
Y_max <- apply(Y, 1, max)
table(W = W, Y_max = Y_max)
```

```
   Y_max
W    0  1
  0 31  0
  1  0 19
```

Fit the logistic regression model to `Y_max`:

```
m2 <- glm(Y_max ~ 1, family = binomial)
coef(m2)
```

```
(Intercept)
 -0.4895482
```

```
plogis(coef(m2))
```

```
(Intercept)
       0.38
```

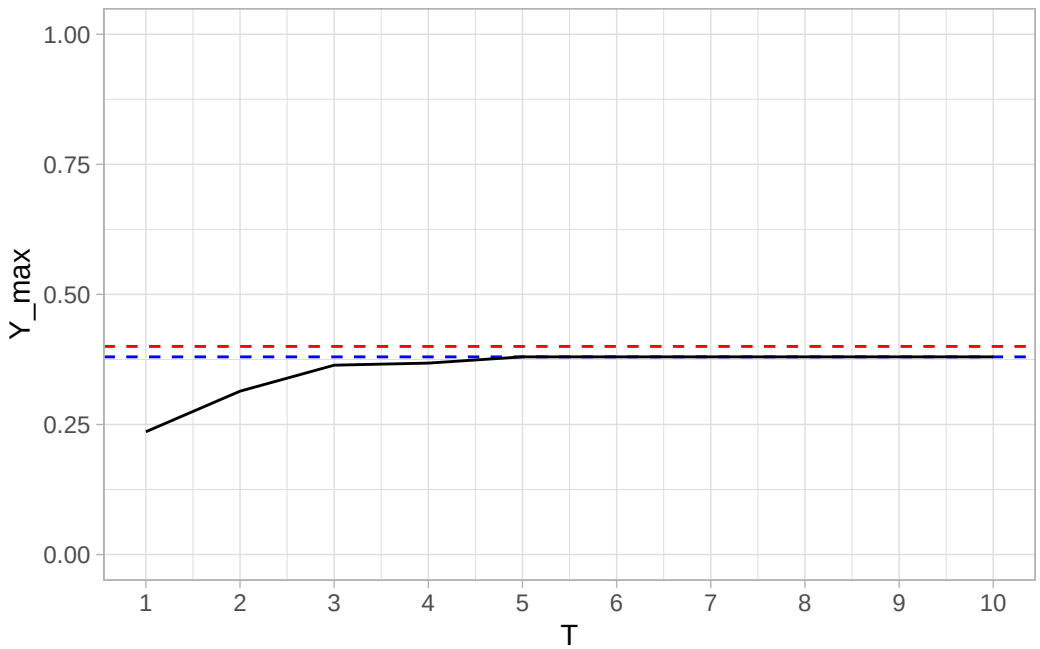
What happens when we increase the number of visits?

Change the value of `T` and compare the `Y_max` to `W`. What happens to our *naive* estimator of using `Y_max`?

The maximum over a large number of visits will approach the latent variable `W`:

```
n2 <- 500
W2 <- rbinom(n = n2, size = 1, prob = phi.true)
Tvals <- 1:10

data.frame(
    T = Tvals,
    Y_max = sapply(Tvals, \(T) {
        mean(apply(
            replicate(T, rbinom(n = n2, size = W, prob = p.true)),
            1, max
        ))
    })
) |> ggplot(aes(x = T, y = Y_max)) +
    geom_hline(yintercept = phi.true, lty = 2, col = "red") +
    geom_hline(yintercept = mean(W), lty = 2, col = "blue") +
    geom_line() +
    ylim(0, 1) +
    scale_x_continuous(breaks = Tvals) +
    theme_light()
```



Fitting occupancy models

The unmarked package implements the multiple-visit occupancy model.

> References
>
> MacKenzie et al., 2002. Estimating site occupancy rates when detection probabilities are less than one. Ecology, 83:2248–2255. Fulltext, DOI 10.1890/0012-9658(2002)083[2248:ESORWD]2.0.CO;2

- Organize the data and an unmarked occupancy data frame
- Fit the model with the `occu`
- the `~1 ~1` formula says that we do not have covariates, just the intercepts

```
umf <- unmarked::unmarkedFrameOccu(y = Y)
summary(umf)
```

```
unmarkedFrame Object

50 sites
Maximum number of observations per site: 5
Mean number of observations per site: 5
```

```
Sites with at least one detection: 19

Tabulation of y observations:
  0   1
186  64
```

```r
m3 <- unmarked::occu(~1 ~ 1, umf)
m3
```

```
Call:
unmarked::occu(formula = ~1 ~ 1, data = umf)

Occupancy (logit-scale):
 Estimate    SE     z P(>|z|)
   -0.483 0.292 -1.65   0.098

Detection (logit-scale):
 Estimate    SE    z P(>|z|)
    0.713 0.222 3.21 0.00134

AIC: 190.2551
Number of sites: 50
```

We use the `plogis()` function again to transform the estimates to the probability scale and compare with our `phi.true` and `p.true` values:

```r
plogis(coef(m3, type = "det"))
```

```
   p(Int)
0.6710909
```

```r
plogis(coef(m3, type = "state"))
```

```
 psi(Int)
0.3814687
```

Maximum likelihood estimator

The `glm()` and `occu()` functions use maximum likelihood to find the parameter estimates for a given data set. In other words, we coefficients (maximum likelihood estimate, or MLE) maximize the likelihood function for the data set in question. The likelihood function can be relatively simple and easy to calculate, or it can be computationally challenging to compute (e.g. for hierarchical or mixed models).

The likelihood function L for the simple occupancy model with parameters p and ϕ for multiple visits data can be written as:

$$L(p, \phi; y_{1,1}, \dots, y_{n,T}) = \prod_{i=1}^{n} \left[\phi \left(\binom{T}{y_{i\cdot}} p^{y_{i\cdot}} (1-p)^{T-y_{i\cdot}} \right) + (1-\phi) I(y_{i\cdot} = 0) \right]$$

where $y_{i\cdot} = \sum_{T}^{t=1} y_{i,t}$ and $I(y_{i\cdot} = 0)$ is an indicator function that is equal to 1 if $y_{i\cdot} = 0$.

Here is the R code to calculate the log likelihood for the occupancy model:

```
L_fun_occu <- function(Y, p, phi) {
    ydot <- rowSums(Y)
    T <- ncol(Y)
    L <- prod(
        phi *
            (choose(T, ydot) * p^ydot * (1 - p)^(T - ydot)) +
            (1 - phi) * (ydot == 0)
    )
    L
}
```

Next, we evaluate the likelihood function at different values of `p` and `phi` while keeping the data `Y` constant. We set up the grid for this using `expand.grid()`:

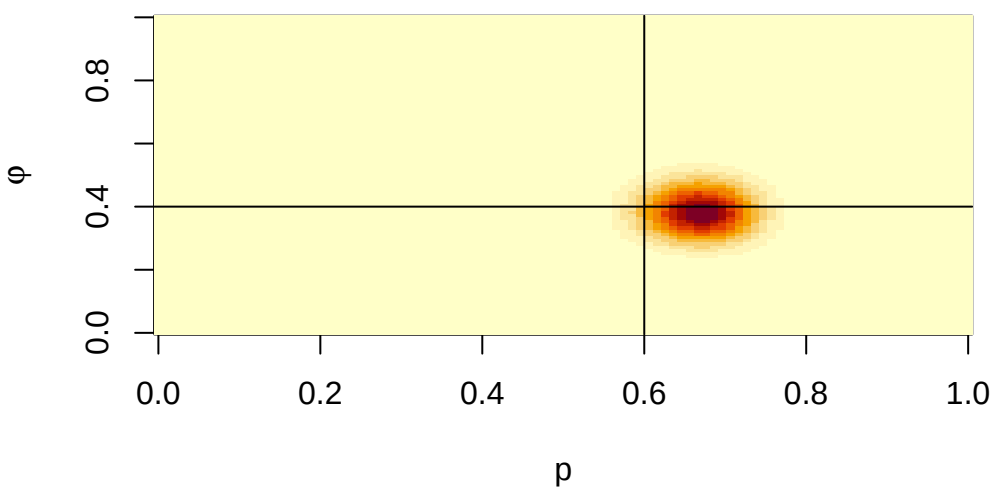
```
g <- 100
grid <- expand.grid(
    p = seq(0, 1, length.out = g),
    phi = seq(0, 1, length.out = g),
    L = NA
)

for (i in 1:nrow(grid)) {
    grid$L[i] <- L_fun_occu(
        Y = Y,
```

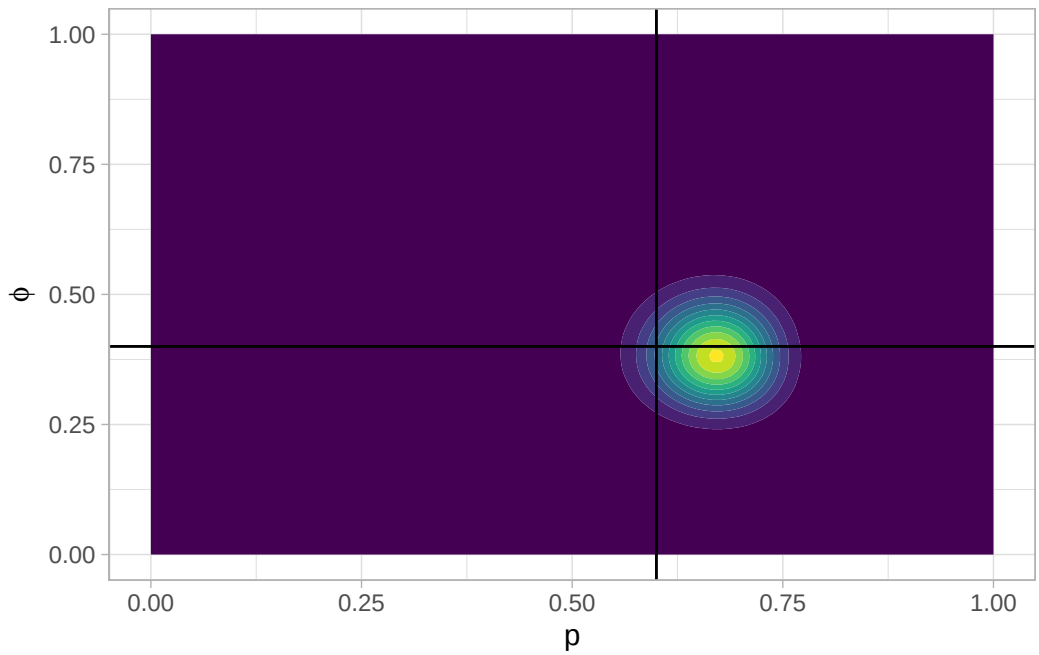
```
        p = grid$p[i],
        phi = grid$phi[i]
    )
}
```

When we plot the likelihood surface, we see the maximum, the lines indicate the true probability values:

```
image(
    list(
        x = unique(grid$p),
        y = unique(grid$phi),
        z = matrix(grid$L, g, g)
    ),
    xlab = "p",
    ylab = expression(varphi)
)
abline(h = phi.true, v = p.true, col = 1, lwd = 1)
```




```
grid |> ggplot(aes(x = p, y = phi, z = L)) +
    geom_contour_filled(show.legend = FALSE) +
    geom_hline(yintercept = phi.true) +
    geom_vline(xintercept = p.true) +
    xlab("p") +
    ylab(expression(phi)) +
    theme_light()
```




```r
grid[which.max(grid$L), ]
```

```
             p       phi            L
3867 0.6666667 0.3838384 2.234106e-27
```

If the rgl package is installed, we can view the surface in 3D:

```r
if (interactive()) {
    library(rgl)

    L_mat <- matrix(grid$L, g, g)
    dcpal_grbu <- colorRampPalette(c("#18bc9c", "#3498db"))
    Col <- rev(dcpal_grbu(12))[cut(L_mat, breaks = 12)]

    open3d()
    persp3d(
        x = unique(grid$p),
        y = unique(grid$phi),
        z = L_mat / max(L_mat),
        col = Col,
        theta = 50, phi = 25, expand = 0.75, ticktype = "detailed",
        xlab = "p", ylab = expression(phi), zlab = "L"
```

```
    )
    quads3d(
        x = rep(p.true, 4),
        y = c(0, 0, 1, 1),
        z = c(0, 1, 1, 0),
        alpha = 0.5, col = 2
    )
    quads3d(
        x = c(0, 0, 1, 1),
        y = rep(phi.true, 4),
        z = c(0, 1, 1, 0),
        alpha = 0.5, col = 4
    )
}
```

> **Note**
>
> We will circle back to these plots later, feel free to explore it with different settings of n, T, p, and ϕ.

Field data analysis

We will use the ABMI data, which has 3 equal (3.3 min) time intervals for each point count visit. We use the point level data, but only for the center point, so we can keep sample size small and run the code quickly. When analyzing such data sets for real, we would use all the data and not subset it like this.

```
if (!interactive()) {
    d <- readRDS("../data/ab_birds_site_year_pt.rds")
} else {
    d <- readRDS("data/ab_birds_site_year_pt.rds")
}

spp <- "yellowrumped_warbler"

x <- data.frame(
    y1 = ifelse(d$visits$visit1[, spp] > 0, 1, 0),
    y2 = ifelse(d$visits$visit2[, spp] > 0, 1, 0),
    y3 = ifelse(d$visits$visit3[, spp] > 0, 1, 0),
    n1 = d$visits$visit1[, spp],
```

```r
    n2 = d$visits$visit2[, spp],
    n3 = d$visits$visit3[, spp],
    d$vars
)
x$yday <- as.POSIXlt(x$fielddate)$yday / 365
x$hr <- x$fieldtime / 24
x$y_max <- pmax(x$y1, x$y2, x$y3)
x$n_max <- pmax(x$n1, x$n2, x$n3)
x <- x |>
    filter(
        hr > 0.1,
        hr < 0.6,
        yday > 0.37,
        yday < 0.5
    )
x0 <- x[x$point == 1, ]
```

Naive occupancy estimates for the 3 visits:

```r
m_v1 <- glm(y1 ~ 1, data = x0, family = binomial)
coef(m_v1)
```

```
(Intercept)
  -1.076354
```

```r
plogis(coef(m_v1))
```

```
(Intercept)
  0.2541966
```

```r
mean(x0$y1)
```

```
[1] 0.2541966
```

```r
m_v2 <- glm(y2 ~ 1, data = x0, family = binomial)
coef(m_v2)
```

```
(Intercept)
  -1.121121
```

```r
plogis(coef(m_v2))
```

```
(Intercept)
  0.2458034
```

```r
mean(x0$y2)
```

```
[1] 0.2458034
```

```r
m_v3 <- glm(y3 ~ 1, data = x0, family = binomial)
coef(m_v3)
```

```
(Intercept)
  -1.147166
```

```r
plogis(coef(m_v3))
```

```
(Intercept)
  0.2410072
```

```r
mean(x0$y3)
```

```
[1] 0.2410072
```

Given that the 3 subsequent 'visits' are actually time intervals within the same point count, we can safely assume that the true occupancy status does not change between the visits (closed population assumption). Therefore, we can use the `y_max` variable to fit a naive occupancy model to approximate the true occupancy status:

```r
m_max <- glm(y_max ~ 1, data = x0, family = binomial)
coef(m_max)
```

```
(Intercept)
 -0.7696873
```

```r
plogis(coef(m_max))
```

```
(Intercept)
  0.3165468
```

```r
mean(x0$y_max)
```

```
[1] 0.3165468
```

Question: What do you think about the closed population assumption? What if visits are days, weeks, months, or years apart?

Let's fit the occupancy model to the data using all 3 visits:

```r
umf <- unmarked::unmarkedFrameOccu(y = x0[, c("y1", "y2", "y3")])
summary(umf)
```

```
unmarkedFrame Object

834 sites
Maximum number of observations per site: 3
Mean number of observations per site: 3
Sites with at least one detection: 264

Tabulation of y observations:
   0    1
1884  618
```

```r
m_mv <- unmarked::occu(~1 ~ 1, umf)
m_mv
```

```
Call:
unmarked::occu(formula = ~1 ~ 1, data = umf)

Occupancy (logit-scale):
 Estimate    SE   z  P(>|z|)
   -0.752 0.075 -10 1.11e-23

Detection (logit-scale):
```

```
 Estimate     SE    z  P(>|z|)
     1.21 0.0897 13.5 1.14e-41

AIC: 1873.258
Number of sites: 834
```

```r
plogis(coef(m_mv, type = "det"))
```

```
   p(Int)
0.7709228
```

```r
plogis(coef(m_mv, type = "state"))
```

```
 psi(Int)
0.3203975
```

Use covariates: for status and detection

```r
umf2 <- unmarked::unmarkedFrameOccu(
    y = x0[, c("y1", "y2", "y3")],
    siteCovs = x0[, c("pconif", "thf")],
    obsCovs = list(
        hr = x0[, c("hr", "hr", "hr")],
        yday = x0[, c("yday", "yday", "yday")]
    )
)
summary(umf2)
```

```
unmarkedFrame Object

834 sites
Maximum number of observations per site: 3
Mean number of observations per site: 3
Sites with at least one detection: 264

Tabulation of y observations:
   0    1
1884  618

Site-level covariates:
```

```
     pconif            thf
 Min.   :0.0000   Min.   :0.00000
 1st Qu.:0.0000   1st Qu.:0.00000
 Median :0.0000   Median :0.06333
 Mean   :0.2740   Mean   :0.34896
 3rd Qu.:0.6036   3rd Qu.:0.86650
 Max.   :1.0000   Max.   :1.00000

Observation-level covariates:
       hr               yday
 Min.   :0.1493   Min.   :0.3890
 1st Qu.:0.2111   1st Qu.:0.4082
 Median :0.2257   Median :0.4192
 Mean   :0.2516   Mean   :0.4273
 3rd Qu.:0.2562   3rd Qu.:0.4493
 Max.   :0.5271   Max.   :0.4822
```

```r
m_mv2 <- unmarked::occu(
    ~ hr + I(hr^2) + yday ~ pconif + I(pconif^2) + thf,
    umf2
)
m_mv2
```

```
Call:
unmarked::occu(formula = ~hr + I(hr^2) + yday ~ pconif + I(pconif^2) +
    thf, data = umf2)

Occupancy (logit-scale):
            Estimate    SE     z  P(>|z|)
(Intercept)    -1.05 0.154 -6.81 1.01e-11
pconif          5.67 0.988  5.74 9.28e-09
I(pconif^2)    -3.97 1.010 -3.93 8.43e-05
thf            -2.18 0.324 -6.74 1.63e-11

Detection (logit-scale):
            Estimate    SE      z P(>|z|)
(Intercept)     5.09  2.25  2.260  0.0239
hr             -5.46  9.85 -0.554  0.5795
I(hr^2)         9.74 15.55  0.626  0.5312
yday           -7.45  4.18 -1.781  0.0749

AIC: 1618.481
```

```
Number of sites: 834

Warning: Large or missing SE values. Be very cautious using these results.
```

```r
m_mv@AIC
```

```
[1] 1873.258
```

```r
m_mv2@AIC
```

```
[1] 1618.481
```

Fitted values reflect expected values of the marginal distribution, i.e. combining the state and detection processes:

```r
summary(fitted(m_mv2))
```

```
       V1                 V2                 V3
 Min.   :0.02592   Min.   :0.02592   Min.   :0.02592
 1st Qu.:0.04205   1st Qu.:0.04205   1st Qu.:0.04205
 Median :0.19161   Median :0.19161   Median :0.19161
 Mean   :0.24616   Mean   :0.24616   Mean   :0.24616
 3rd Qu.:0.47832   3rd Qu.:0.47832   3rd Qu.:0.47832
 Max.   :0.59388   Max.   :0.59388   Max.   :0.59388
```

```r
colMeans(umf2@y)
```

```
       y1        y2        y3
0.2541966 0.2458034 0.2410072
```

We can calculate the expected occupancy probability for each site by using the coefficients of the model matrix:

```r
f <- m_mv2@formula[-2]
f
```

```
~pconif + I(pconif^2) + thf
```

```r
X <- model.matrix(f, x0)
head(X)
```

```
            (Intercept)    pconif I(pconif^2)        thf
1_2009_1              1 0.6440748   0.4148324 0.00000000
1001_2011_1           1 1.0000000   1.0000000 0.74507161
1002_2011_1           1 0.8230018   0.6773319 0.12879228
1003_2011_1           1 0.9614613   0.9244079 0.03853867
1007_2008_1           1 0.9644391   0.9301428 0.03556088
1008_2008_1           1 0.6248039   0.3903800 0.17998054
```

```r
coef(m_mv2, type = "state")
```

```
        psi(Int)      psi(pconif) psi(I(pconif^2))         psi(thf)
       -1.048719         5.674886        -3.972183        -2.184805
```

```r
X[1, ] * coef(m_mv2, type = "state")
```

```
(Intercept)      pconif I(pconif^2)         thf
  -1.048719    3.655051   -1.647790    0.000000
```

```r
sum(X[1, ] * coef(m_mv2, type = "state"))
```

```
[1] 0.9585418
```

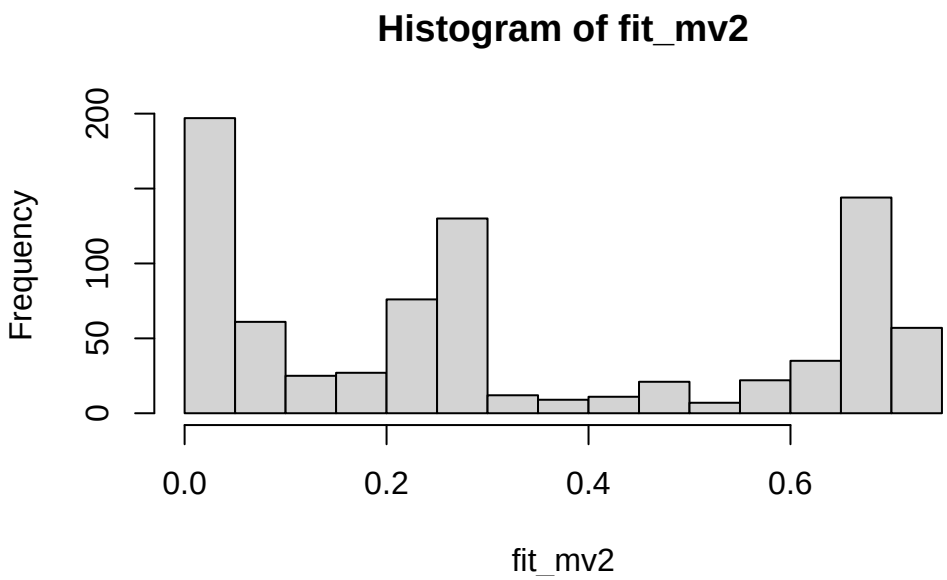
```r
head(X %*% coef(m_mv2, type = "state"), 1)
```

```
              [,1]
1_2009_1 0.9585418
```

```r
fit_mv2 <- plogis(X %*% coef(m_mv2, type = "state"))
summary(fit_mv2)
```

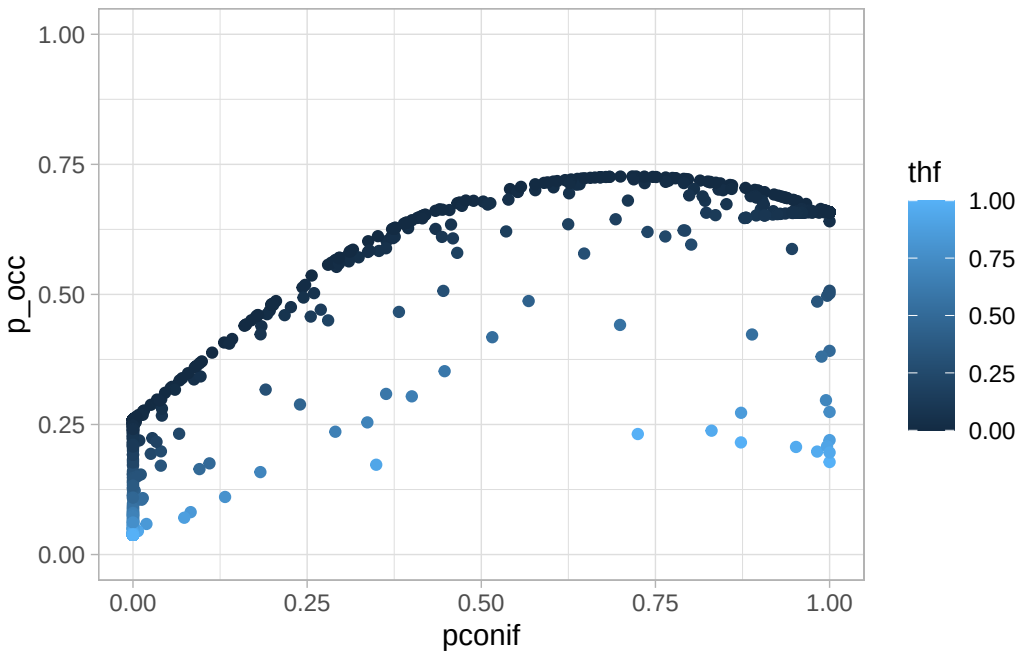
```
       V1
 Min.   :0.03792
 1st Qu.:0.05665
 Median :0.25650
 Mean   :0.32101
 3rd Qu.:0.64404
 Max.   :0.72674
```

```r
hist(fit_mv2)
```

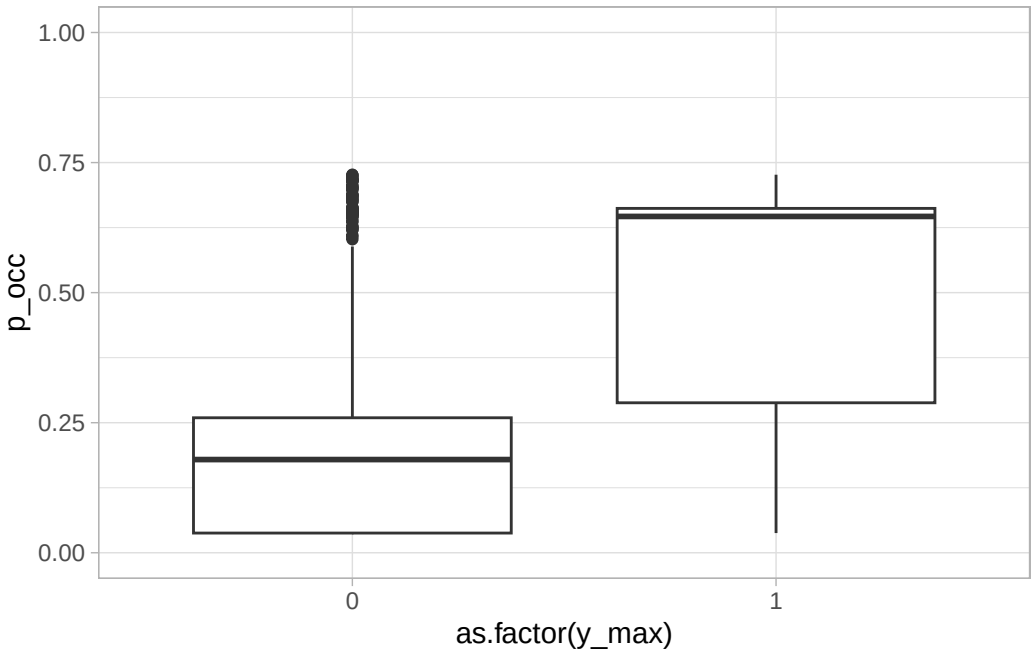


Visualize the fitted occupancy probabilities with respect to the covariates:

```r
data.frame(x0, p_occ = fit_mv2) |>
    ggplot(aes(x = pconif, y = p_occ, col = thf)) +
    geom_point() +
    ylim(0, 1)
```



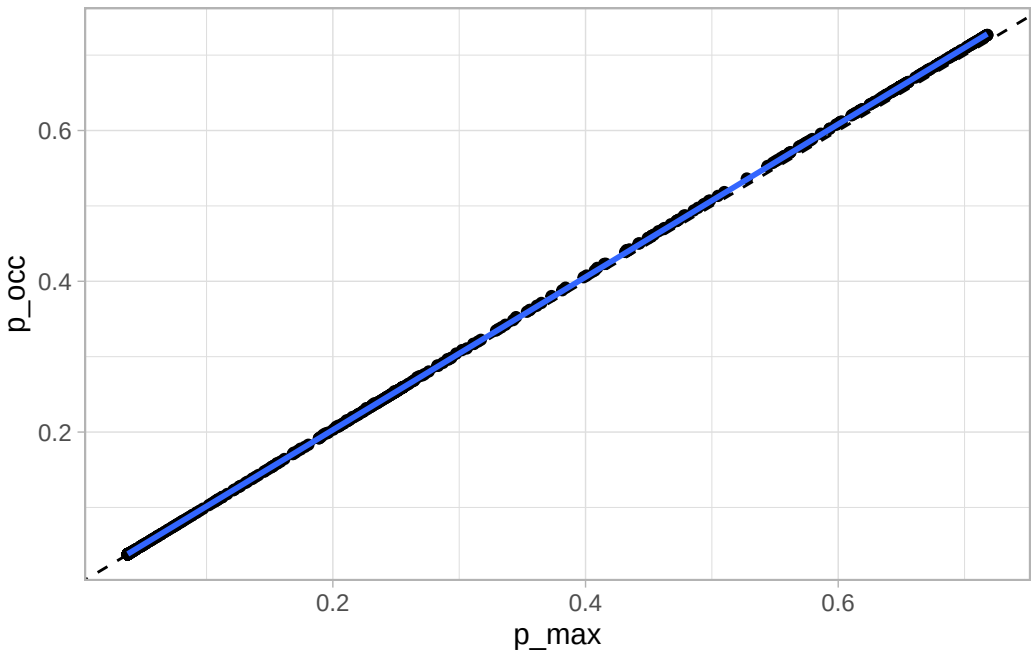

```
data.frame(x0, p_occ = fit_mv2) |>
    ggplot(aes(x = as.factor(y_max), y = p_occ)) +
    geom_boxplot() +
    ylim(0, 1)
```



```r
m_max2 <- glm(y_max ~ pconif + I(pconif^2) + thf, data = x0, family = binomial)

data.frame(p_occ = fit_mv2, p_max = fitted(m_max2)) |>
    ggplot(aes(x = p_max, y = p_occ)) +
    geom_point() +
    geom_abline(lty = 2) +
    geom_smooth(method = "lm", se = FALSE)
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```r
mean(fit_mv2 / fitted(m_max2))
```

```
[1] 1.012492
```

Predict detection probability given detection covariates:

```r
Z <- model.matrix(m_mv2@formula[-3], x0)
head(Z)
```

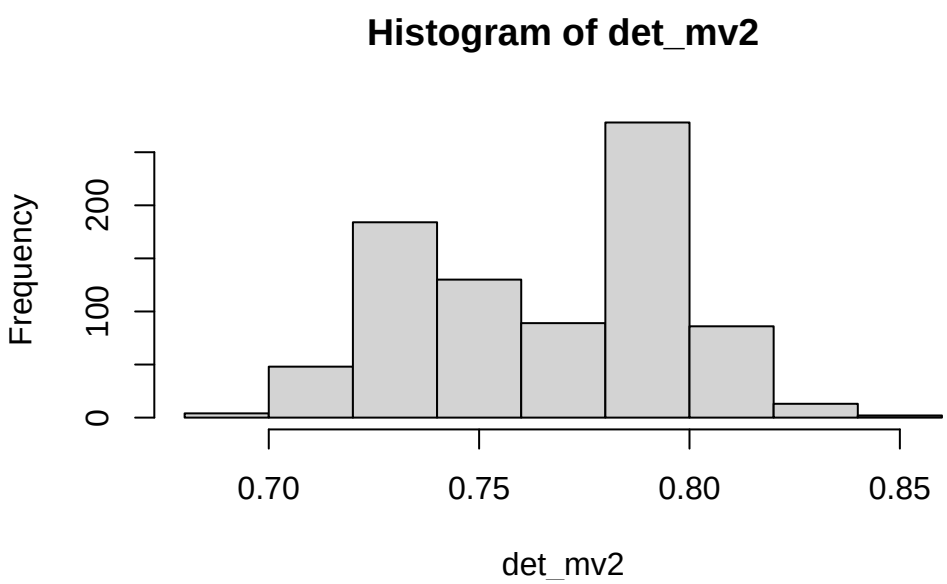
```
            (Intercept)        hr    I(hr^2)      yday
1_2009_1              1 0.2354167 0.05542101 0.4410959
```

```
1001_2011_1           1 0.2437500 0.05941406 0.4465753
1002_2011_1           1 0.2465278 0.06077595 0.4356164
1003_2011_1           1 0.3888889 0.15123457 0.4383562
1007_2008_1           1 0.2666667 0.07111111 0.4054795
1008_2008_1           1 0.2020833 0.04083767 0.4164384
```

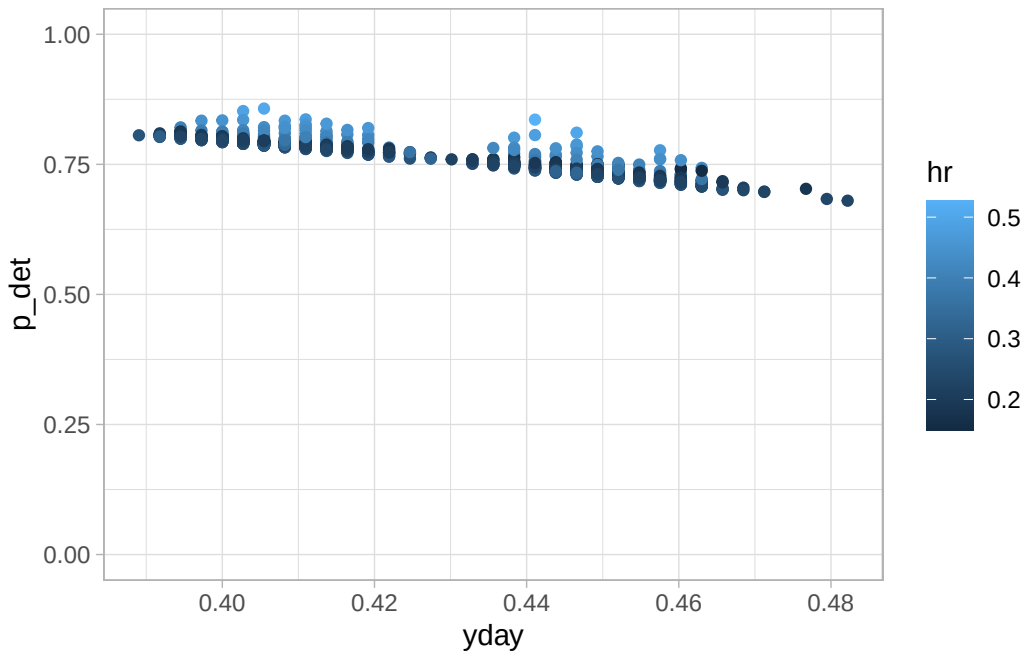
```r
det_mv2 <- plogis(Z %*% coef(m_mv2, type = "det"))
summary(det_mv2)
```

```
       V1
 Min.   :0.6802
 1st Qu.:0.7368
 Median :0.7761
 Mean   :0.7661
 3rd Qu.:0.7923
 Max.   :0.8572
```

```r
hist(det_mv2)
```




```r
data.frame(x0, p_det = det_mv2) |>
    ggplot(aes(x = yday, y = p_det, col = hr)) +
    geom_point() +
    ylim(0, 1)
```



Question: How would we assess model fit? Can we use AUC/ROC?

What do we get when only using 2 visits?

```
umf_v2 <- unmarked::unmarkedFrameOccu(y = x0[, c("y1", "y2")])
m_mv_v2 <- unmarked::occu(~1 ~ 1, umf_v2)
m_mv_v2
```

```
Call:
unmarked::occu(formula = ~1 ~ 1, data = umf_v2)

Occupancy (logit-scale):
 Estimate     SE     z  P(>|z|)
   -0.849 0.0777 -10.9 8.01e-28

Detection (logit-scale):
 Estimate    SE    z  P(>|z|)
     1.62 0.142 11.4 5.66e-30

AIC: 1396.047
Number of sites: 834
```

```r
plogis(coef(m_mv_v2, type = "det"))
```

```
   p(Int)
0.8345324
```

```r
plogis(coef(m_mv_v2, type = "state"))
```

```
psi(Int)
0.299569
```

How about only using 1 visit?

```r
umf_v1 <- unmarked::unmarkedFrameOccu(y = x0[, c("y1"), drop = FALSE])
m_mv_v1 <- unmarked::occu(~1 ~ 1, umf_v1)
m_mv_v1
```

```
Call:
unmarked::occu(formula = ~1 ~ 1, data = umf_v1)

Warning in sqrt(diag(vcov(obj, fixedOnly = fixedOnly))): NaNs produced

Occupancy (logit-scale):
 Estimate  SE   z P(>|z|)
   0.0167 NaN NaN     NaN

Warning in sqrt(diag(vcov(obj, fixedOnly = fixedOnly))): NaNs produced

Detection (logit-scale):
 Estimate  SE   z P(>|z|)
   0.0167 NaN NaN     NaN

AIC: 949.5873
Number of sites: 834

Warning in sqrt(diag(v)): NaNs produced

Warning: Large or missing SE values. Be very cautious using these results.
```

```r
plogis(coef(m_mv_v1, type = "det"))
```

```
   p(Int)
0.5041792
```

```r
plogis(coef(m_mv_v1, type = "state"))
```

```
 psi(Int)
0.5041792
```

Something is wrong here, right? Let put this aside for now, we will come back to this later.